Lab 8 - Edge Detection

Sajad Abdulkarim Alsalem | Image Processing

Objective and selected image

Apply Sobel, Prewitt, and Canny edge detection and compare three Canny threshold pairs. The selected image is the scikit-image coffee photograph, different from kid1.jpg in the reference. Run the notebook from top to bottom; no upload is needed.

Setup and preprocessing

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data

# New image: coffee photograph (RGB), not the reference image
img = data.coffee()
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

# Use the same preprocessing for all methods and threshold pairs
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

Task 1 - Sobel Edge Detector

```
sobel_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
sobel_magnitude = cv2.magnitude(sobel_x, sobel_y)
sobel_edges = cv2.normalize(
    sobel_magnitude, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)
```

The horizontal and vertical derivatives are combined into a gradient magnitude. Normalization avoids overflow when converting the display map to uint8.

Task 1 - Prewitt Edge Detector

```
kernel_x = np.array([
    [-1, 0, 1], [-1, 0, 1], [-1, 0, 1]
], dtype=np.float64)
kernel_y = np.array([
    [-1, -1, -1], [0, 0, 0], [1, 1, 1]
], dtype=np.float64)

prewitt_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
prewitt_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)
prewitt_magnitude = cv2.magnitude(prewitt_x, prewitt_y)
prewitt_edges = cv2.normalize(
    prewitt_magnitude, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)
```

Task 1 - Canny Edge Detector

```
canny_edges = cv2.Canny(blurred, 50, 150)
```

Display code

```
fig, ax = plt.subplots(2, 2, figsize=(12, 8))
images = [img, sobel_edges, prewitt_edges, canny_edges]
titles = ["Original Coffee Image", "Sobel", "Prewitt", "Canny (50, 150)"]

for a, result, title in zip(ax.flat, images, titles):
    if result.ndim == 3:
        a.imshow(result)
    else:
        a.imshow(result, cmap="gray", vmin=0, vmax=255)
    a.set_title(title)
    a.axis("off")

plt.tight_layout()
plt.show()
```

Sobel and Prewitt show gradient strength and are normalized separately. Canny produces thin binary edges. The same Gaussian-blurred grayscale input is used for all three detectors.

Task 1 - Original image and edge maps

Original Coffee Image



Sobel



Prewitt



Canny (50, 150)



Sobel and Prewitt show similar major outlines with softer edge responses. Canny produces thinner contours and suppresses many weaker responses. Separate normalization means map brightness cannot be used to compare raw Sobel and Prewitt magnitudes.

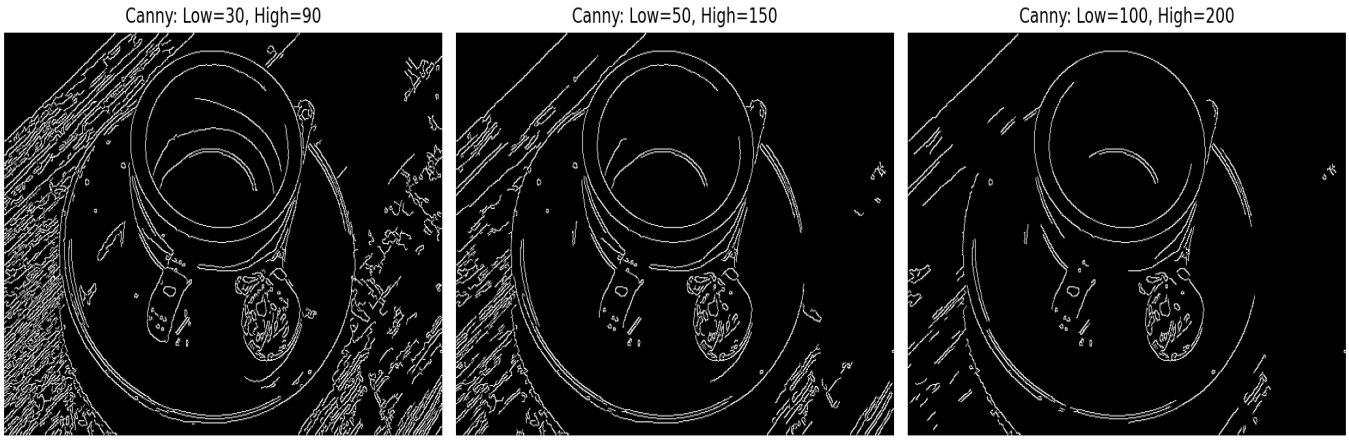
Task 2 - Effect of Canny thresholds

```
threshold_pairs = [(30, 90), (50, 150), (100, 200)]
canny_results = []
edge_counts = []

fig, ax = plt.subplots(1, 3, figsize=(15, 5))
for a, (low, high) in zip(ax, threshold_pairs):
    edges = cv2.Canny(blurred, low, high)
    canny_results.append(edges)
    edge_counts.append(int(np.count_nonzero(edges)))
    a.imshow(edges, cmap="gray", vmin=0, vmax=255)
    a.set_title(f"Canny: Low={low}, High={high}")
    a.axis("off")

plt.tight_layout()
plt.show()

print("Low   High   Edge pixels")
for (low, high), count in zip(threshold_pairs, edge_counts):
    print(f"{low:<5} {high:<6} {count}")
```



Edge pixels: (30, 90) = 24,319; (50, 150) = 13,823; (100, 200) = 7,790.

Lower thresholds retain more weak edges and fine texture. The middle pair removes some weaker responses while preserving major outlines. Higher thresholds retain fewer, stronger boundaries and lose faint details. Strong candidates exceed the high threshold; weaker candidates above the low threshold survive only when connected to strong edges.

Conclusion

Sobel and Prewitt produce edge-strength maps. Canny produces thin binary contours. Increasing the Canny thresholds reduces detected edges and fine details, while decreasing them reveals more texture and may include unwanted responses.